

Pivot Index / Equilibrium Index

1. What is Pivot Index?

A **Pivot Index** is an index where the sum of all elements on the **left** is equal to the sum of all elements on the **right**.

The element at the pivot itself is **not included** in either side.

LEFT		PIVOT		RIGHT
↓		↓		↓
[1 2 3]		6		3 2 1]

For pivot index 3:

Left = 1 + 2 + 3 = 6
Right = 3 + 2 + 1 = 6

Since:

Left == Right

index 3 is the **Pivot / Equilibrium Index**.

2. Why is it called Equilibrium?

Think of a balance:

LEFT SIDE = RIGHT SIDE

Both sides have the same total, so the array is **balanced** at that index.

That is why it is also called an **Equilibrium Index**.

3. The Main Idea

For every index `i`, think like this:

```
[ elements before i ] [ arr[i] ] [ elements after i ]  
      LEFT           PIVOT      RIGHT
```

We need to check:

```
LEFT SUM == RIGHT SUM
```

If true, `i` is a pivot index.

4. Left-Right Approach

A simple way to solve the problem is to keep track of:

- `left_sum` = sum of elements before the current index
- `right_sum` = sum of elements after the current index
- `total_sum` = sum of the complete array

First calculate the total sum.

Then move from left to right.

At index `i`:

```
left_sum  = sum before i  
right_sum = total_sum - left_sum - arr[i]
```

Then check:

```
if left_sum == right_sum:  
    i is the pivot index
```

5. Most Important Formula

Right Sum Formula

```
right_sum = total_sum - left_sum - arr[i]
```

Why?

The total sum contains three parts:

```
TOTAL = LEFT + PIVOT + RIGHT
```

So:

```
RIGHT = TOTAL - LEFT - PIVOT
```

Since the pivot element is `arr[i]`:

```
right_sum = total_sum - left_sum - arr[i]
```

6. Step-by-Step Example

Consider:

```
arr = [1, 2, 3, 6, 3, 2, 1]
```

Total sum:

```
1 + 2 + 3 + 6 + 3 + 2 + 1 = 18
```

So:

```
TOTAL = 18
```

Start with:

```
left_sum = 0
```

Index 0

```
arr[0] = 1
```

```
left = 0
```

```
right = 18 - 0 - 1 = 17
```

```
0 != 17
```

Not a pivot.

After checking index 0, add its value to the left side:

```
left_sum = 0 + 1 = 1
```

Index 1

```
arr[1] = 2
```

```
left = 1
```

```
right = 18 - 1 - 2 = 15
```

```
1 != 15
```

Not a pivot.

Update:

```
left_sum = 1 + 2 = 3
```

Index 2

```
arr[2] = 3
```

```
left = 3
```

```
right = 18 - 3 - 3 = 12
```

```
3 != 12
```

Not a pivot.

Update:

```
left_sum = 3 + 3 = 6
```

Index 3

```
arr[3] = 6
```

```
left = 6
```

```
right = 18 - 6 - 6 = 6
```

Now:

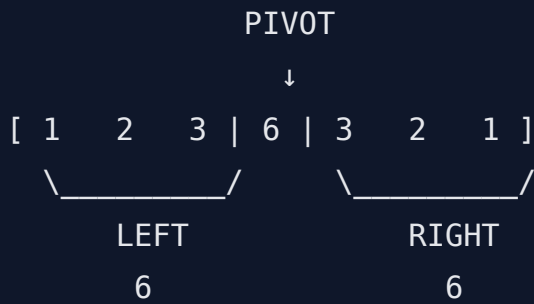
```
left == right
```

```
6 == 6
```

So:

```
Pivot Index = 3
```

7. Visual Understanding



The pivot element `6` is **not included** in either sum.

```
LEFT  = 1 + 2 + 3 = 6
```

```
RIGHT = 3 + 2 + 1 = 6
```

Therefore index `3` is the equilibrium point.

8. Why Use This Approach?

A simple method would calculate the entire left and right sum again for every index.

That can repeat a lot of work.

Instead, we calculate the total once and maintain `left_sum` while moving through the array.

At every index:

```
left_sum  -> already known
arr[i]    -> current value
right_sum -> calculated using the formula
```

So we can check each index in constant work.

9. Simple Algorithm

1. Calculate `total_sum` of the array.
2. Set `left_sum = 0`.
3. Go through every index `i`.
4. Calculate:
`right_sum = total_sum - left_sum - arr[i]`
5. Check:
`if left_sum == right_sum`
then `i` is the pivot index.
6. Add `arr[i]` to `left_sum`.
7. Continue until the array ends.

10. Python Code

```
class Solution:
    def pivotIndex(self, nums):
        total_sum = sum(nums)
        left_sum = 0

        for i in range(len(nums)):
            right_sum = total_sum - left_sum - nums[i]

            if left_sum == right_sum:
                return i

            left_sum += nums[i]

        return -1
```

11. Code Flow in Simple Words

Suppose we are at index `i`.

Step 1

We already know the sum of everything before `i`:

```
left_sum
```

Step 2

We know the sum of the whole array:

```
total_sum
```

Step 3

Remove the left side:

```
total_sum - left_sum
```

Now the remaining part contains:

```
PIVOT + RIGHT
```

Step 4

Remove the pivot element:

```
total_sum - left_sum - arr[i]
```

Now only the right side remains.

Therefore:

```
right_sum = total_sum - left_sum - arr[i]
```

12. Important Detail: Update Left After Checking

This order is important:

```
right_sum = total_sum - left_sum - nums[i]

if left_sum == right_sum:
    return i

left_sum += nums[i]
```

Why do we add `nums[i]` **after** checking?

Because the current element is the **pivot**.

The pivot must not be included in the left or right sum.

So:

```
Before checking i:
left_sum = elements before i

Current element:
arr[i] = pivot

Right side:
elements after i
```

Only after checking do we move the current element into the left side for the next index.

13. Quick Formula to Remember

The whole problem can be reduced to these three lines:

```
TOTAL = sum of all elements

RIGHT = TOTAL - LEFT - arr[i]

if LEFT == RIGHT:
    i is Pivot Index
```

14. Complexity

```
Time:  O(n)
Space: O(1)
```

We go through the array once, and we only use a few variables.

15. Use this pattern when the problem asks:

- Find an index where left sum = right sum
- Find the equilibrium index
- Find the pivot index
- Find a balance point in an array
- Find an index where removing/ignoring that element makes the sum on both sides equal

16. Final Mental Model

Whenever you see **Pivot Index / Equilibrium Index**, think:

```

        i
        ↓
[  LEFT  |  arr[i]  |  RIGHT  ]

LEFT SUM == RIGHT SUM

```

And use:

```
right_sum = total_sum - left_sum - arr[i]
```

Then:

```
if left_sum == right_sum:
    pivot found
```

One-line memory trick

Total = Left + Pivot + Right

Therefore:

Right = Total - Left - Pivot